

Documento técnico de implementación y evidencias de rendimiento: Ecommify

Optimización de bases de datos relacionales y documentales en PostgreSQL/Supabase y

MongoDB Atlas

Fredy Orlando Pulido Quintero

Myriam Andrea Martínez Fontecha

Nicolás Felipe Torres Amaya

Juan Francisco Javier Pérez Rivero

Proyecto Ecommify

Junio de 2026

Resumen ejecutivo

Este documento presenta la implementación técnica y las evidencias cuantitativas del ejercicio de optimización de bases de datos del proyecto Ecommify. La solución integró PostgreSQL en Supabase para el modelo transaccional y analítico relacional, con PostGIS para búsquedas geoespaciales, y MongoDB Atlas para colecciones documentales orientadas a consulta y lectura flexible. La implementación usó los datos de Olist, transformados y cargados mediante notebooks de preparación, carga y benchmark.

Las principales optimizaciones aplicadas fueron: particionamiento por rango de fechas en la tabla orders; índices GiST para consultas geoespaciales; índices GIN para JSONB, arreglos y búsquedas trigram; índices BRIN y B-tree para columnas temporales y relacionales; una vista materializada de ventas por categoría y mes; y, en MongoDB, índices simples, compuestos y 2dsphere para consultas documentales y geoespaciales.

Los resultados evidencian mejoras relevantes: en PostgreSQL, Q3 PostGIS redujo el tiempo de 134.52 ms a 8.21 ms, equivalente a una mejora de 93.90%; Q4 con trigram pasó de 86.70 ms a 8.96 ms, una mejora de 89.67%; y Q2 de join transaccional pasó de 601.67 ms a 156.86 ms, una mejora de 73.93%. En MongoDB, la consulta geoespacial pasó de un escenario no ejecutable eficientemente sin índice 2dsphere a un efficiency ratio óptimo de 1.0, mientras que Q1 de reviews mejoró de 1.73 a 1.0.

La evidencia generada en el notebook de benchmarks incluye archivos CSV con métricas antes/después, planes JSON completos y la gráfica comparativa de rendimiento. El README de evidencias define que en PostgreSQL el indicador principal es el tiempo en milisegundos de EXPLAIN ANALYZE, y que en MongoDB se usa el ratio docs_examined / docs_returned, donde 1.0 es el valor óptimo.

Contenido

1. Implementación PostgreSQL
2. Implementación MongoDB
3. Evidencias cuantitativas de rendimiento
4. Sincronización entre sistemas
5. Lecciones aprendidas
6. Conclusiones

Referencias

Apéndices

1. Implementación PostgreSQL

La implementación relacional se ejecutó en Supabase PostgreSQL. El notebook

01_pg_supabase_setup.ipynb habilitó extensiones, creó tipos compuestos, tablas maestras, tablas transaccionales, particiones, vistas materializadas, triggers e índices. Se habilitaron pgcrypto, PostGIS, pg_trgm y btree_gin para soportar UUID, consultas geoespaciales, búsqueda por similitud y estructuras de indexación avanzadas.

1.1 Scripts DDL ejecutados en Supabase

El modelo incluyó tipos compuestos address y dimensions; tablas maestras geolocation, product_category_name_translation, customers, sellers, products y warehouses; tablas transaccionales orders, order_items y order_payments; particiones orders_2016, orders_2017, orders_2018 y orders_default; una vista materializada mv_sales_by_category_monthly; y triggers de actualización de campos updated_at.

Tabla 1
Resumen del modelo PostgreSQL implementado.

Objeto	Rol	Diseño aplicado
geolocation	Maestra geográfica	PK zip_code_prefix; location como GEOGRAPHY generada; CHECK de latitud/longitud.
customers	Maestra de clientes	FK a geolocation por zip_code_prefix; UUID como PK; estado y dirección compuesta.
sellers	Maestra de vendedores	UUID como PK; atributos de ubicación del vendedor.
products	Maestra de productos	FK a categorías; dimensions compuesto; JSONB specifications; arrays photos/tags.
orders	Transaccional particionada	PK compuesta order_id + timestamp; FK a customers; CHECK de status; partición RANGE por fecha.
order_items	Detalle transaccional	PK compuesta con item; FK a orders, products y sellers; CHECK de cantidades y valores.
order_payments	Pagos	PK compuesta; FK a orders; CHECK de tipo, secuencia, cuotas y valor > 0.
mv_sales_by_category_monthly	Analítica	MV agregada por mes y categoría para acelerar consultas de ventas.

Nota. El diseño combina integridad relacional, tipos avanzados, particionamiento temporal y vista materializada para consultas analíticas.

1.2 Estrategia de indexación y justificación técnica

La estrategia usó índices distintos según el patrón de acceso. PostgreSQL soporta B-tree, Hash, GiST, SP-GiST, GIN y BRIN; cada uno está diseñado para clases de predicados diferentes. En el proyecto se usaron B-tree para igualdad y joins, GiST para geografía, GIN para JSONB, arrays y trigram, y BRIN para rangos temporales correlacionados con el orden físico de las filas (PostgreSQL Global Development Group, 2026a).

Tabla 2
Estrategia de indexación PostgreSQL.

Objeto	Indice / restriccion	Tipo	Columnas / expresion	Justificacion tecnica
geolocation	geolocation_pkey	PRIMARY KEY / B-tree implicito	zip_code_prefix	Garantiza unicidad del prefijo postal y soporta joins con customers.zip_code_prefix.
geolocation	idx_geo_location_gist	GiST espacial	location	Optimiza consultas PostGIS con ST_DWithin y ST_Distance; fue clave para la mejora de Q3 geoespacial.
product_category_name_translation	product_category_name_translation_pkey	PRIMARY KEY / B-tree implicito	product_category_name	Garantiza unicidad de categorías y soporta la relacion con products.product_category_name.
customers	customers_pkey	PRIMARY KEY / B-tree implicito	customer_id	Identificador tecnico principal del cliente; soporta joins OLTP con orders.customer_id.
customers	customers_customer_unique_id_key	UNIQUE / B-tree implicito	customer_unique_id	Garantiza unicidad del cliente original del dataset.
customers	idx_customers_state	B-tree	customer_state	Mejora filtros y analisis por estado del cliente.
customers	idx_customers_unique	B-tree	customer_unique_id	Indice adicional sobre

Objeto	Indice / restriccion	Tipo	Columnas / expresion	Justificacion tecnica
				identificador unico de negocio; puede considerarse redundante frente al UNIQUE implicito.
customers	idx_customers_zip_code_prefix	B-tree	zip_code_prefix	Mejora el join geoespacial entre customers y geolocation; evita escaneo completo de clientes al resolver clientes cercanos a una ubicacion.
sellers	sellers_pkey	PRIMARY KEY / B- tree implicito	seller_id	Identificador tecnico principal del vendedor.
sellers	idx_sellers_state	B-tree	seller_state	Mejora filtros y agrupaciones por estado del vendedor.
products	products_pkey	PRIMARY KEY / B- tree implicito	product_id	Identificador tecnico principal del producto; soporta joins con order_items.
products	idx_products_specs_gin	GIN	specifications jsonb_path_ops	Optimiza busquedas sobre documentos JSONB de especificaciones del producto.
products	idx_products_tags_gin	GIN	tags	Optimiza consultas sobre arreglos de etiquetas.
products	idx_products_cat_trgm	GIN + trigram	product_category_name gin_trgm_ops	Optimiza busquedas aproximadas o por similitud sobre categorias; impacto directo en Q4 TRGM.
products	idx_products_updated	B-tree	updated_at	Mejora consultas incrementales, auditoria y procesos ETL basados en fecha de actualizacion.
warehouses	warehouses_pkey	PRIMARY KEY / B- tree	warehouse_id	Identificador tecnico principal del deposito.

Objeto	Indice / restriccion	Tipo	Columnas / expresion	Justificacion tecnica
		implicito		
warehouses	idx_warehouses_name	B-tree	name	Optimiza la busqueda del deposito Deposito SP Centro usado en la consulta geoespacial Q3.
warehouses	idx_warehouses_location_gist	GiST espacial	location	Prepara la tabla de depositos para busquedas espaciales futuras y joins espaciales bidireccionales.
orders	orders_pkey / indices PK por particion	PRIMARY KEY compuest o / B-tree implicito	order_id, order_purchase_timestam p	Garantiza unicidad de ordenes en tabla particionada y soporta relaciones compuestas desde order_items y order_payments.
orders	idx_orders_customer	B-tree	customer_id	Mejora joins entre orders y customers.
orders	idx_orders_updated	B-tree	updated_at	Soporta consultas incrementales y procesos ETL por fecha de actualizacion.
orders	idx_orders_purchase_brin	BRIN	order_purchase_timestam p	Optimiza rangos de fechas sobre tabla particionada, con bajo costo de almacenamiento; apoya Q1 y Q2.
order_items	order_items_pkey	PRIMARY KEY compuest o / B-tree implicito	order_id, order_purchase_timestam p, order_item_id	Garantiza unicidad de items por orden y soporta integridad con orders.
order_items	idx_items_product	B-tree	product_id	Mejora joins y agregaciones por producto.
order_items	idx_items_seller	B-tree	seller_id	Mejora joins y analisis por vendedor.
order_payments	order_payments_pkey	PRIMARY KEY	order_id, order_purchase_timestam	Garantiza unicidad de pagos por orden y soporta integridad

Objeto	Indice / restriccion	Tipo	Columnas / expresion	Justificacion tecnica
		compuest o / B-tree implicito	p, payment_sequential	con orders.
order_payments	idx_payments_type	B-tree	payment_type	Mejora analisis por medio de pago.
mv_sales_by_category_monthly	uidx_mv_sales_cat_month	UNIQUE / B-tree	year_month, category	Garantiza unicidad de ventas por mes y categoria; permite refresco concurrente de la vista materializada y mejora Q5.
mv_sales_by_category_monthly	idx_mv_sales_month_category_revenue	B-tree con INCLUDE	year_month, category INCLUDE gross_revenue, gross_sales, orders_count, avg_item_price	Indice cubriendo para consultas analiticas de ventas por periodo y categoria; reduce lecturas sobre la vista materializada.

Nota. El índice idx_customers_zip_code_prefix fue identificado durante el análisis de Q3 porque PostgreSQL no crea automáticamente índices para columnas FK.

1.3 Particionamiento aplicado

La tabla orders fue particionada por rango sobre order_purchase_timestamp. Se crearon particiones anuales para 2016, 2017 y 2018, además de una partición default. El objetivo fue permitir partition pruning en consultas con predicados temporales, evitando escaneos sobre años no requeridos. La evidencia README indica que en Q1 el partition pruning fue visible al escanear solo orders_2018.

1.4 Queries críticas optimizadas

Las consultas críticas fueron: Q1 rango de fecha sobre orders; Q2 join OLTP entre orders, customers y order_payments; Q3 geoespacial PostGIS para clientes cercanos a Deposito SP Centro; Q4 búsqueda textual por similitud con pg_trgm; y Q5 agregación de ventas por categoría sobre vista materializada.

En Q3, el plan antes de optimizar incluía escaneos secuenciales sobre geolocation y customers. Después de los índices, el plan usó idx_geo_location_gist para ST_DWithin y idx_customers_zip_code_prefix para resolver el join por zip_code_prefix. Esto redujo el tiempo de 134.52 ms a 8.21 ms, con mejora de 16.38x.

Tabla 3
PostgreSQL: comparación antes/después.

Query	Antes	Después	Mejora	Reducción
Q1 rango fecha	0.40 ms	0.20 ms	1.96x	48.87%
Q2 join oltp	601.67 ms	156.86 ms	3.84x	73.93%
Q3 postgis	134.52 ms	8.21 ms	16.38x	93.90%
Q4 trgm	86.70 ms	8.96 ms	9.68x	89.67%
Q5 mv ventas	0.80 ms	0.52 ms	1.54x	35.16%

Nota. Tiempos tomados de EXPLAIN ANALYZE. Menor tiempo es mejor.

Tabla 4
Evidencia de planes PostgreSQL.

Query	Plan antes	Índices antes	Plan después	Índices después
Q1 rango fecha	Limit, Seq Scan	Sin índice explícito en el plan	Limit, Seq Scan	Sin índice explícito en el plan
Q2 join oltp	Limit, Aggregate, Sort, Nested Loop, Hash Join, Seq Scan	customers_pkey	Limit, Aggregate, Sort, Nested Loop, Hash Join, Seq Scan	customers_pkey
Q3 postgis	Limit, Hash Join, Nested Loop, Seq Scan, Hash	Sin índice explícito en el plan	Limit, Nested Loop, Seq Scan, Index Scan	idx_geo_location_gist, idx_customers_zip_code_prefix
Q4 trgm	Limit, Sort, Seq Scan	Sin índice explícito en el plan	Limit, Sort, Bitmap Heap Scan, Bitmap Index Scan	idx_products_cat_trgm
Q5 mv_ventas	Limit, Sort, Aggregate, Seq Scan	Sin índice explícito en el plan	Limit, Sort, Aggregate, Index Scan	uidx_mv_sales_cat_month

Nota. Resumen derivado de explain_plans_20260616_000208.json.

1.5 Evidencia gráfica PostgreSQL y MongoDB

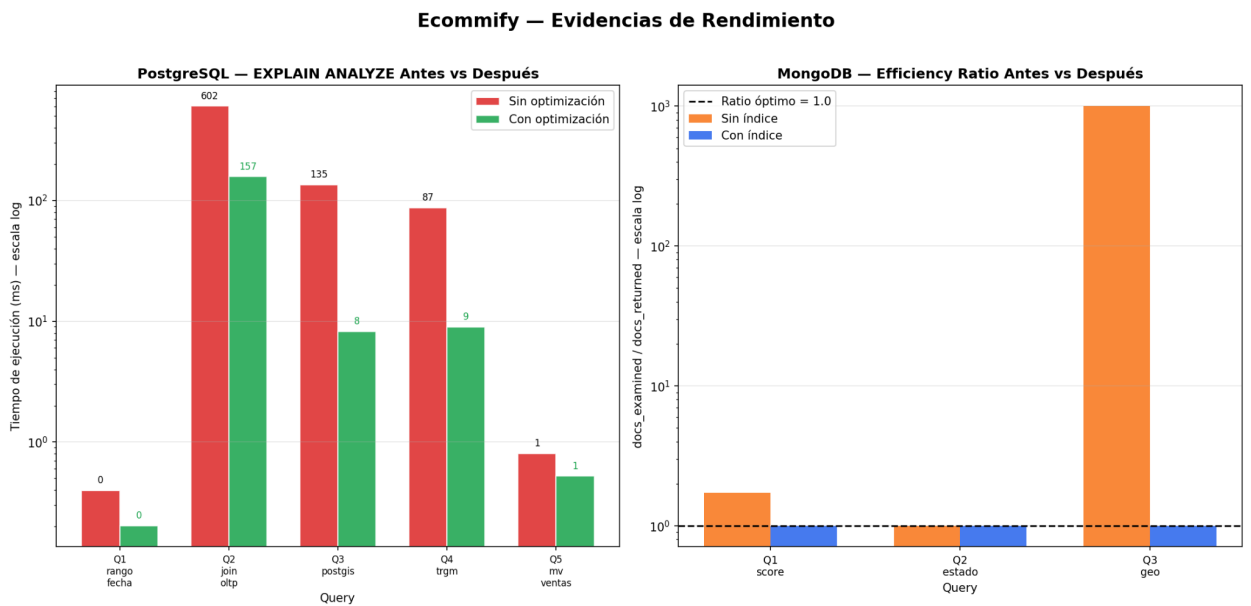


Figura 1. Evidencias de rendimiento PostgreSQL y MongoDB antes/después.

2. Implementación MongoDB

La implementación documental se realizó en MongoDB Atlas con tres colecciones principales: geolocation, reviews y orders_summary. La colección geolocation almacenó documentos GeoJSON Point; reviews almacenó reseñas con score, comentario y fechas; y orders_summary se construyó mediante un flujo ETL desde PostgreSQL, denormalizando pedidos, cliente, ítems y datos de pago para consultas rápidas de lectura.

Tabla 5
Colecciones y esquemas documentales MongoDB.

Colección	Estructura documental	Uso principal
geolocation	{ zip_code_prefix, city, state, location: { type: "Point", coordinates: [lng, lat] } }	Consultas geoespaciales con \$near.
reviews	{ review_id, order_id, review_score, review_comment_title, review_comment_message, dates }	Consultas por score y búsqueda textual.
orders_summary	{ _id: order_id, status, purchase_date, customer, items[], payment_total, payment_type_main, review_score, etl_updated_at }	Lectura agregada/denormalizada desde PostgreSQL.

Nota. El diseño favorece lecturas frecuentes y evita joins en tiempo de consulta.

2.1 Índices implementados con justificación

MongoDB permite índices sobre campos de documentos, índices compuestos y geoespaciales.

En este ejercicio se usaron índices en review_score, status, purchase_date, customer.state, etl_updated_at, el compuesto customer.state + status y el índice 2dsphere en location. El método explain() fue usado para evaluar etapas COLLSCAN, IXSCAN, FETCH y

GEO_NEAR_2DSPHERE, junto con executionStats (MongoDB, Inc., 2026a, 2026b).

Tabla 6
Índices MongoDB implementados.

Coleccion	Indice	Tipo	Campos	Justificacion tecnica
Todas las colecciones	_id_	B-tree implicito	_id	Indice creado automaticamente por MongoDB; garantiza acceso directo por

Coleccion	Indice	Tipo	Campos	Justificacion tecnica
				identificador de documento.
geolocation	location_2dsphere	2dsphere	location	Optimiza consultas geoespaciales con \$near y \$maxDistance; fue determinante para la mejora de Q3 geo.
geolocation	zip_code_prefix_1	B-tree unico	zip_code_prefix	Garantiza unicidad del prefijo postal y permite busquedas directas por codigo postal.
reviews	review_id_1	B-tree unico	review_id	Garantiza unicidad del identificador de resena proveniente del dataset.
reviews	order_id_1	B-tree	order_id	Soporta busquedas y posibles cruces logicos entre resenas y ordenes.
reviews	review_score_1	B-tree	review_score	Optimiza consultas por calificacion; en Q1 paso de COLLSCAN a IXSCAN, reduciendo documentos examinados.
reviews	review_comment_message_text_review_comment_title_text	Text index	review_comment_message, review_comment_title	Permite busquedas textuales sobre comentarios y

Coleccion	Indice	Tipo	Campos	Justificacion tecnica
				titulos de resenas en idioma portugues.
orders_summary	status_1	B-tree	status	Mejora filtros por estado de la orden, por ejemplo delivered.
orders_summary	purchase_date_-1	B-tree descendente	purchase_date	Optimiza consultas recientes, ordenamientos y rangos por fecha de compra.
orders_summary	customer.state_1	B-tree	customer.state	Mejora consultas por estado del cliente.
orders_summary	etl_updated_at_1	B-tree	etl_updated_at	Soporta trazabilidad y procesos incrementales de sincronizacion o recarga ETL.
orders_summary	customer.state_1_status_1	B-tree compuesto	customer.state, status	Optimiza consultas combinadas por estado del cliente y estado de la orden; fue usado en Q2 para mejorar selectividad y eficiencia del plan.

Nota. El índice 2dsphere fue crítico: sin él, la consulta \$near no es viable como búsqueda geoespacial eficiente.

2.2 Aggregation pipelines optimizados

La optimización documental se apoyó en una decisión de modelado: construir `orders_summary` como documento agregado desde PostgreSQL. Este flujo reemplaza joins repetitivos por documentos denormalizados que incorporan estado del pedido, cliente, ítems y pagos. Para pipelines futuros, la recomendación es iniciar con `$match` sobre campos indexados, reducir el conjunto con `$project`, agrupar solo cuando sea necesario y evitar etapas bloqueantes antes de filtrar.

2.3 Diseño teórico de sharding y replica sets

Para alta disponibilidad, MongoDB replica datos mediante replica sets; para escalabilidad horizontal, distribuye datos mediante sharding. En un crecimiento productivo de Ecommify se recomienda un replica set por shard y sharding selectivo en colecciones de alto volumen como `orders_summary` o `geolocation`, eligiendo claves que distribuyan carga y preserven patrones de consulta (MongoDB, Inc., 2026c, 2026d).

- `orders_summary`: candidato a shard key compuesta o hashed basada en `_id/order_id`, con índices secundarios por estado, fecha y `customer.state` para analítica regional.
- `reviews`: candidato a sharding por `order_id` o hash de `review_id` si el volumen crece y se requiere distribución uniforme.
- `geolocation`: requiere cuidado en sharding geoespacial; se sugiere mantener índice `2dsphere` y evaluar patrones de consulta por región antes de fragmentar.
- Replica sets: al menos tres nodos por conjunto para failover automático y separación de lecturas, según criticidad del escenario.

Tabla 7**MongoDB: efficiency ratio antes/después.**

Query	Ratio antes	Ratio después	Mejora ratio
Q1_score	1.73	1.00	1.73x
Q2_estado	1.00	1.00	1.00x
Q3_geo	999.00	1.00	999.00x

Nota. Efficiency ratio = docs_examined / docs_returned. El valor óptimo es 1.0.

Tabla 8**Evidencia explain() MongoDB.**

Query	ms antes	docs antes	stage antes	índice antes	ms después	docs después	stage después	índice después
Q1_score	88	98410	COLLSCAN	Sin índice / no ejecutable sin índice	70	56910	FETCH, IXSCAN	review_score_1
Q2_estado	199	78178	FETCH, IXSCAN	customer.state_1_status_1	121	78178	FETCH, IXSCAN	customer.state_1_status_1
Q3_geo	No ejecutable				3	152	GEO_NEAR_2DSPHERE, FETCH, IXSCAN	location_2dsphere

Nota. Resumen derivado de executionStats y winningPlan.

3. Evidencias cuantitativas de mejoras de rendimiento

La evidencia cuantitativa consolida dos tipos de indicadores. En PostgreSQL se midió tiempo de ejecución con EXPLAIN ANALYZE, que permite observar nodos de plan, costos, tiempos, bloques leídos y uso de índices (PostgreSQL Global Development Group, 2026b). En MongoDB se midió executionTimeMillis y efficiency ratio. Un ratio 1.0 significa que cada documento examinado fue retornado, reduciendo lecturas innecesarias.

Las mejoras más significativas se observaron en consultas donde la optimización cambió el método de acceso: Q3 PostGIS pasó de Seq Scan y Hash Join a Index Scan GiST + Index Scan B-tree; Q4 cambió de Seq Scan a Bitmap Index Scan con idx_products_cat_trgm; Q5 cambió de Seq Scan sobre la MV a Index Scan sobre uidx_mv_sales_cat_month. En MongoDB, Q3_geo pasó a usar GEO_NEAR_2DSPHERE con location_2dsphere.

- Q3 PostGIS: mejora de 16.38x y reducción de 93.90%; impacto alto por búsqueda espacial y join geográfico.
- Q4 trigram: mejora de 9.68x y reducción de 89.67%; impacto alto por búsqueda textual aproximada.

- Q2 join OLTP: mejora de 3.84x y reducción de 73.93%; impacto relevante sobre lectura relacional transaccional.
- Q5 MV ventas: mejora de 1.54x; la vista materializada ya reduce el costo base de agregación, y el índice mejora el filtro por fecha.
- MongoDB Q3_geo: mejora teórica de 999x en ratio por pasar de escenario sin índice geoespacial a 2dsphere óptimo.

4. Sincronización entre sistemas

El flujo de sincronización implementado fue un ETL desde PostgreSQL hacia MongoDB.

PostgreSQL mantuvo la fuente de verdad transaccional para orders, customers, order_items y order_payments, mientras MongoDB recibió una proyección denormalizada en orders_summary.

El campo etl_updated_at permitió identificar el momento de actualización del documento.

La estrategia de consistencia fue eventual: los datos se consolidan en PostgreSQL y posteriormente se sincronizan hacia MongoDB por lotes mediante bulk_write y upsert. Este enfoque evita duplicados por _id = order_id y permite regenerar la colección documental sin afectar el modelo relacional. Para un escenario productivo, el flujo podría evolucionar hacia CDC, colas de eventos o procesos incrementales por updated_at.

5. Lecciones aprendidas

5.1 Obstáculos encontrados y soluciones aplicadas

- La identificación de nuevos índices fue un desafío central. Aunque la DDL inicial incluía índices relevantes, el análisis de Q3 reveló la necesidad de idx_customers_zip_code_prefix para soportar eficientemente la relación geolocation -> customers.

- El manejo de llaves foráneas durante la carga requirió validar datos antes de insertar, especialmente en customers, products, sellers, orders, order_items y order_payments. La solución recomendada fue validar por SQL mediante staging tables y joins contra tablas destino.
- Los CHECK constraints protegieron la calidad de datos. Un caso relevante fue order_payments.payment_value = 0.00, que violaba la regla payment_value > 0. La carga debió filtrar registros inválidos antes de insertarlos.
- Para simular el escenario “antes”, fue necesario eliminar índices adicionales no obligatorios, preservando PRIMARY KEY, UNIQUE, FOREIGN KEY y CHECK. PostgreSQL no ofrece un mecanismo general para apagar y encender índices como estrategia estándar; el enfoque seguro fue DROP/CREATE de índices adicionales.
- La interpretación de EXPLAIN requiere observar no solo el tiempo, sino también el tipo de nodo, el índice seleccionado, los bloques compartidos y los cambios de plan.

5.2 Limitaciones del free tier y workarounds implementados

El uso de Supabase para PostgreSQL y MongoDB Atlas facilitó la creación de cuentas y la disponibilidad de entornos administrados sin aprovisionamiento manual de infraestructura. Esta facilidad permitió concentrar el ejercicio en modelado, carga, índices y benchmark. Sin embargo, las cuentas gratuitas presentaron limitaciones de recursos, acceso y ancho de red; en varias ejecuciones se observaron timeouts y latencias variables propias de ambientes compartidos o de bajo dimensionamiento.

Como workaround, se aumentó el statement_timeout de PostgreSQL hasta 10 minutos en las conexiones de carga y benchmark, por ejemplo options="-c statement_timeout=600000".

También se usaron cargas por lotes, commit explícito, rollback ante errores transaccionales, reducción de operaciones simultáneas y medición repetible mediante notebooks separados.

6. Conclusiones

El ejercicio demuestra que la optimización efectiva requiere combinar diseño físico, diseño lógico y medición. En PostgreSQL, los mayores beneficios provinieron de índices especializados y particionamiento: GiST resolvió el patrón geoespacial, GIN/trigram resolvió la búsqueda aproximada, BRIN y particiones soportaron rangos temporales, y la vista materializada redujo el costo de agregaciones. En MongoDB, los índices simples, compuestos y 2dsphere redujeron escaneos innecesarios y transformaron consultas geoespaciales en operaciones viables.

La relación entre PostgreSQL y MongoDB se justificó por propósito: PostgreSQL como fuente transaccional e integridad relacional, y MongoDB como capa documental de lectura, exploración y consulta flexible. El resultado cuantitativo respalda la estrategia: los tiempos de PostgreSQL disminuyeron de forma relevante en las consultas críticas, y MongoDB alcanzó ratios óptimos en los escenarios indexados.

Referencias

MongoDB, Inc. (2026a). `Cursor.explain()` (mongosh method). MongoDB Manual.

<https://www.mongodb.com/docs/manual/reference/method/cursor.explain/>

MongoDB, Inc. (2026b). 2dsphere indexes. MongoDB Manual.

<https://www.mongodb.com/docs/manual/core/indexes/index-types/geospatial/2dsphere/>

MongoDB, Inc. (2026c). Replication. MongoDB Manual.

<https://www.mongodb.com/docs/manual/replication/>

MongoDB, Inc. (2026d). Sharding. MongoDB Manual.

<https://www.mongodb.com/docs/manual/sharding/>

PostGIS Project Steering Committee. (2026). `ST_DWithin`. PostGIS documentation.

https://postgis.net/docs/ST_DWithin.html

PostgreSQL Global Development Group. (2026a). Index types. PostgreSQL 18 documentation.

<https://www.postgresql.org/docs/current/indexes-types.html>

PostgreSQL Global Development Group. (2026b). Using EXPLAIN. PostgreSQL 18

documentation. <https://www.postgresql.org/docs/current/using-explain.html>

Supabase. (2026). Pricing & fees. <https://supabase.com/pricing>

Supabase. (2026). Production checklist. Supabase Docs.

<https://supabase.com/docs/guides/deployment/going-into-prod>

Ecommify. (2026). Evidencias de rendimiento: PostgreSQL EXPLAIN ANALYZE y MongoDB `explain()` [Conjunto de archivos de benchmark].

Apéndice A. Scripts DDL principales PostgreSQL

A continuación se resumen fragmentos representativos de la DDL ejecutada. Los scripts completos se encuentran en el notebook 01_pg_supabase_setup.ipynb.

A.1 Tabla geolocation e índice GiST

```
CREATE TABLE IF NOT EXISTS geolocation (  
    zip_code_prefix INTEGER PRIMARY KEY,  
    geolocation_lat DOUBLE PRECISION NOT NULL CHECK (geolocation_lat BETWEEN -33.75 AND 5.27),  
    geolocation_lng DOUBLE PRECISION NOT NULL CHECK (geolocation_lng BETWEEN -73.99 AND -34.73),  
    geolocation_city VARCHAR(60) NOT NULL,  
    geolocation_state CHAR(2) NOT NULL,  
    location GEOGRAPHY(POINT, 4326) GENERATED ALWAYS AS  
        (ST_MakePoint(geolocation_lng, geolocation_lat)::geography) STORED  
);  
CREATE INDEX IF NOT EXISTS idx_geo_location_gist ON geolocation USING GIST (location);
```

A.2 Tabla orders particionada

```
CREATE TABLE IF NOT EXISTS orders (  
    order_id UUID NOT NULL,  
    customer_id UUID NOT NULL REFERENCES customers(customer_id) ON DELETE RESTRICT,  
    order_status VARCHAR(20) NOT NULL CHECK (order_status IN  
  
    ('delivered', 'shipped', 'canceled', 'approved', 'processing', 'unavailable', 'invoiced', 'created')),  
    order_purchase_timestamp TIMESTAMPTZ NOT NULL,  
    PRIMARY KEY (order_id, order_purchase_timestamp)  
) PARTITION BY RANGE (order_purchase_timestamp);  
  
CREATE TABLE IF NOT EXISTS orders_2016 PARTITION OF orders  
    FOR VALUES FROM ('2016-01-01') TO ('2017-01-01');  
CREATE TABLE IF NOT EXISTS orders_2017 PARTITION OF orders  
    FOR VALUES FROM ('2017-01-01') TO ('2018-01-01');  
CREATE TABLE IF NOT EXISTS orders_2018 PARTITION OF orders  
    FOR VALUES FROM ('2018-01-01') TO ('2019-01-01');  
CREATE TABLE IF NOT EXISTS orders_default PARTITION OF orders DEFAULT;
```

A.3 Vista materializada de ventas

```
CREATE MATERIALIZED VIEW IF NOT EXISTS mv_sales_by_category_monthly AS  
SELECT date_trunc('month', o.order_purchase_timestamp) AS year_month,  
    COALESCE(pcnt.product_category_name_english, 'sin_categoria') AS category,  
    COUNT(DISTINCT o.order_id) AS orders_count,  
    SUM(oi.price) AS gross_sales,  
    SUM(oi.price + oi.freight_value) AS gross_revenue,  
    AVG(oi.price) AS avg_item_price  
FROM orders o  
JOIN order_items oi ON oi.order_id = o.order_id  
    AND oi.order_purchase_timestamp = o.order_purchase_timestamp  
JOIN products p ON p.product_id = oi.product_id  
LEFT JOIN product_category_name_translation pcnt  
    ON pcnt.product_category_name = p.product_category_name  
WHERE o.order_status = 'delivered'  
GROUP BY 1, 2  
WITH NO DATA;
```

```
CREATE UNIQUE INDEX IF NOT EXISTS uidx_mv_sales_cat_month
ON mv_sales_by_category_monthly (year_month, category);
```

Apéndice B. Índices adicionales para optimización

```
CREATE INDEX IF NOT EXISTS idx_customers_zip_code_prefix
ON customers(zip_code_prefix);

CREATE INDEX IF NOT EXISTS idx_warehouses_name
ON warehouses(name);

CREATE INDEX IF NOT EXISTS idx_warehouses_location_gist
ON warehouses USING GIST(location);

CREATE INDEX IF NOT EXISTS idx_mv_sales_month_category_revenue
ON mv_sales_by_category_monthly (year_month, category)
INCLUDE (gross_revenue, gross_sales, orders_count, avg_item_price);
```

Apéndice C. Consulta PostGIS optimizada

```
WITH deposito AS (
    SELECT location
    FROM warehouses
    WHERE name = 'Deposito SP Centro'
    LIMIT 1
)
SELECT c.customer_id,
       c.customer_state,
       ST_Distance(g.location, d.location) / 1000.0 AS dist_km
FROM deposito d
JOIN geolocation g
  ON ST_DWithin(g.location, d.location, 50000)
JOIN customers c
  ON c.zip_code_prefix = g.zip_code_prefix
LIMIT 100;
```